

API Operacional: Consulta de Eventos em Tempo Real e Autenticação

Autenticação

Todos os endpoints exigem os seguintes headers:

Header	Descrição
<code>Authorization</code>	<code>Bearer <token></code> — JWT obtido via endpoint de autenticação
<code>x-company-id</code>	UUID do tenant (company)

O header `x-company-id` é validado pelo `AuthMiddleware` em todas as requisições. Qualquer tentativa de acessar dados de outro tenant resulta em `403 Forbidden`.

Endpoints

`GET /operational/accesses/by-gate`

Retorna a quantidade de check-ins agrupados por portão e setor.

Query Parameters

Parâmetro	Tipo	Obrigatório	Descrição
<code>eventId</code>	string (UUID)	Sim	ID do evento
<code>dateFrom</code>	string (YYYY-MM-DD)	Não	Data inicial do filtro de check-in
<code>dateTo</code>	string (YYYY-MM-DD)	Não	Data final do filtro de check-in

Lógica

- Valida que o `eventId` pertence ao tenant informado via `x-company-id`.
- Se o evento pertencer a um grupo (`eventGroup`), expande automaticamente para todos os eventos do grupo.
- Busca tickets com `checkInStatus = true` agrupados por `gate_id` e `sector_id`.

4. O nome do portão é resolvido via join com a tabela `gates` usando o `gate_template` do evento e o `integration_code` do portão. Se não houver correspondência, retorna o `gate_id` bruto como fallback.
5. **Fallback:** se nenhum ticket com `checkInStatus = true` for encontrado (tenants que não utilizam esse campo), consulta tickets cujo token consta na tabela `check_in_log` via subquery `EXISTS`. O nome do portão segue a mesma lógica de resolução via `gates`.

Exemplo de requisição

```
1 curl -X GET "https://api.bepass.app/operational/accesses/by-gate?eventId"
-H "Authorization: Bearer {token}" \
-H "x-company-id: {companyId}"
```

Com filtro de data

```
1 curl -X GET "https://api.bepass.app/operational/accesses/by-gate?eventId"
-H "Authorization: Bearer {token}" \
-H "x-company-id: {companyId}"
```

Exemplo de resposta

```
1 {
  "data": [
    { "gate": "Portão Norte", "sector": "Setor A", "total": 1240 },
    { "gate": "Portão Sul", "sector": "Setor B", "total": 980 }
  ],
  "total": 2
}
```

Quando o fallback via `check_in_log` é acionado, `sector` retorna `null` pois essa tabela não possui informação de setor.

GET /operational/tickets/total

Retorna o total de ingressos do evento com filtro opcional de status.

Query Parameters

Parâmetro	Tipo	Obrigatório	Descrição
eventId	string (UUID)	Sim	ID do evento
status	string	Não	active (padrão), canceled, all

Lógica

1. Valida que o `eventId` pertence ao tenant e resolve `eventGroup` se aplicável.
2. Executa `COUNT(*)` na tabela `tickets` filtrando pelos `eventIds` resolvidos.
3. Aplica filtro de `status` quando informado. Se omitido, usa `active` como padrão.

Exemplo de requisição

```
1 | # Total de ingressos ativos (padrão)
```

```
curl -X GET "https://api.bepass.app/operational/tickets/total?eventId={eventId}" \
```

```
-H "Authorization: Bearer {token}" \
```

```
-H "x-company-id: {companyId}"
```

Total incluindo cancelados

```
curl -X GET "https://api.bepass.app/operational/tickets/total?eventId={eventId}&status=all" \
```

```
-H "Authorization: Bearer {token}" \
```

```
-H "x-company-id: {companyId}"
```

Exemplo de resposta

```
1 | { "total": 45320 }
```

GET /operational/tickets/by-sector

Retorna a quantidade de ingressos agrupados por setor e portão.

Query Parameters

Parâmetro	Tipo	Obrigatório	Descrição
eventId	string (UUID)	Sim	ID do evento
status	string	Não	active (padrão), canceled, all

Lógica

1. Valida que o `eventId` pertence ao tenant e resolve `eventGroup` se aplicável.
2. Agrupa tickets por `sector_id` e `gate_id`.
3. O nome do portão é resolvido via join com `gates` por `gate_template` + `integration_code`. Se não houver match, retorna o `gate_id` bruto.
4. Aplica filtro de `status` quando informado.

Exemplo de requisição

```
1 curl -X GET "https://api.bepass.app/operational/tickets/by-sector?eventId"
-H "Authorization: Bearer {token}" \
-H "x-company-id: {companyId}"
```

Exemplo de resposta

```
1 {
  "data": [
    { "sector": "Setor Norte", "gate": "Portão 1", "total": 3200 },
    { "sector": "Setor Sul", "gate": "Portão 2", "total": 2800 }
  ],
  "total": 2
}
```

GET /operational/cards/supervisors

Retorna os cartões de supervisores ativos vinculados ao tenant.

Query Parameters

Parâmetro	Tipo	Obrigatório	Descrição
eventGroupId	string (UUID)	Não	ID do grupo de eventos para filtrar supervisores

Lógica

A tabela `cards_supervisor` é global e não possui coluna de tenant. O isolamento é feito cruzando com a tabela `cards`, que vincula cartões a eventos, e eventos a tenants via `company_id`.

1. Se `eventGroupId` for informado, valida que todos os eventos do grupo pertencem ao tenant.
2. Se não informado, busca todos os `eventIds` que possuem cartões vinculados ao tenant via `join cards → events → company_id`.
3. Consulta a tabela `cards` filtrando pelos `eventIds` do tenant com `type = 'supervisor'` e `status = 'active'`.
4. Com os `cardCodes` encontrados, busca os dados completos na tabela `cards_supervisor`.

Exemplo de requisição

```
1 | # Todos os supervisores do tenant
```

```
curl -X GET "https://api.bepass.app/operational/cards/supervisors" \
-H "Authorization: Bearer {token}" \
-H "x-company-id: {companyId}"
```

Exemplo de resposta

```
1 | {
```

```
"data": [
```

```
  { "cardCode": "SUP-001", "name": "João Silva", "type":
    "supervisor", "status": "active" },
```

```
{ "cardCode": "SUP-002", "name": "Maria Santos", "type":  
"supervisor", "status": "active" }  
  
],  
  
"total": 2  
  
}
```

Regras de Negócio

Isolamento por tenant

O header `x-company-id` é validado em todas as requisições pelo `AuthMiddleware`. Um token de um tenant não consegue acessar dados de outro — qualquer tentativa retorna `403 Forbidden`.

Eventos agrupados

Alguns clientes possuem múltiplos registros de evento para um mesmo jogo (ex: setores cadastrados com IDs distintos). Nesses casos, basta informar qualquer `eventId` do grupo — o sistema verifica se o evento possui `eventGroup` e expande automaticamente para todos os eventos vinculados. Isso se aplica a todos os endpoints que recebem `eventId`.

Banco de dados compartilhado

Alguns tenants compartilham a mesma instância de banco de dados PostgreSQL. O isolamento é garantido pela coluna `company_id` presente em todas as queries — nunca por separação de banco.

Resolução do nome do portão

O `gate_id` nos tickets é um código de integração. O nome legível é resolvido via join com a tabela `gates`, filtrando pelo `gate_template` do evento. O campo `integration_code` em `gates` pode conter um único valor ou uma lista separada por vírgula — o sistema trata ambos os casos. Se não houver correspondência, o `gate_id` bruto é retornado como fallback.

Códigos de Resposta

Código	Descrição
200	Sucesso
400	Parâmetros inválidos ou ausentes

401	Token ausente ou inválido
403	Recurso não pertence ao tenant informado
404	Evento não encontrado